

REPRESENTASI KATA dalam NLP

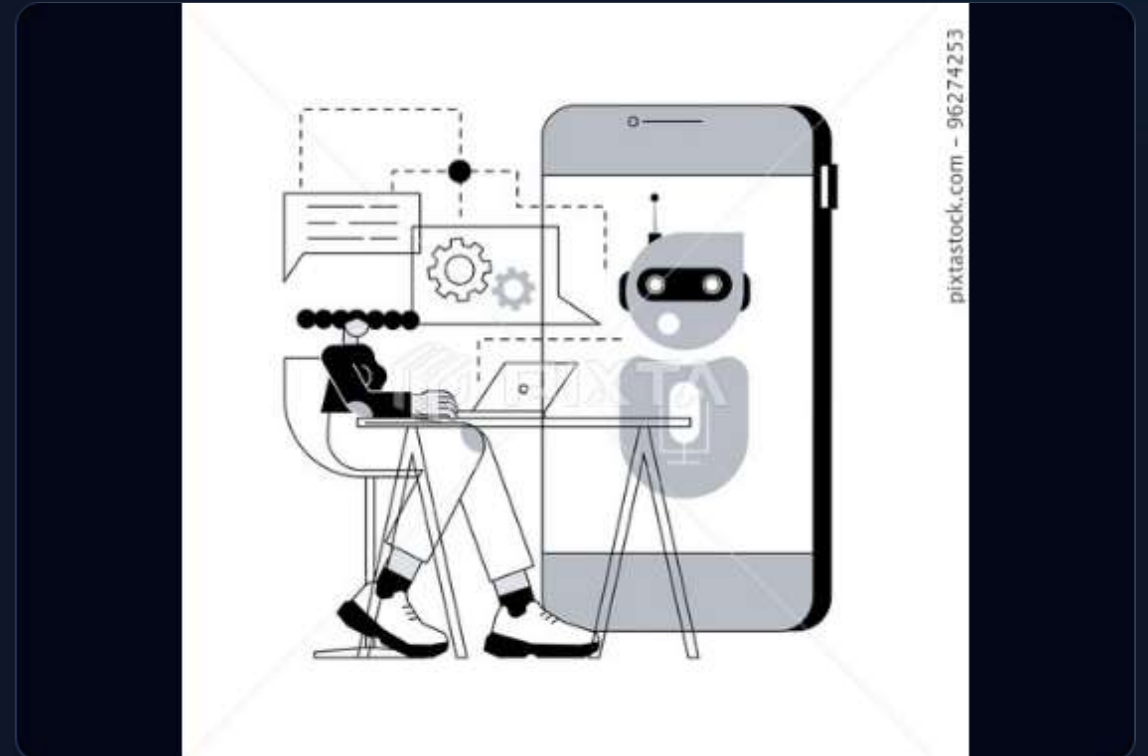
MULAAB

Prinsip Dasar Word Encoding

Dalam bidang **Natural Language Processing (NLP)**, algoritma AI/ML tidak dapat bekerja secara langsung pada data teks mentah.

Kunci pemrosesan teks adalah mengubah kata menjadi representasi angka yang dapat dipahami oleh mesin:

- ✓ **Konversi Teks:** Mengubah kata-kata menjadi numerik agar dapat dikonsumsi model AI.
- ✓ **Representasi Vektor:** Setiap kata diwakili dalam struktur matematis teratur.
- ✓ **Metode Populer:** Label/Integer Encoding dan One-Hot Vector Encoding.



Apa Itu One-Hot Vector Encoding?

Konsep Dasar

Metode ini mengonversi satu kata tunggal menjadi **vektor berdimensi N**, di mana **N** adalah ukuran total kosakata (vocabulary size) dalam korpus.

Dibutuhkan daya komputasi yang relatif kecil untuk mengonversi data teks dan sangat mudah diimplementasikan.

Struktur Vektor One-Hot

Vektor diisi dengan nilai **nol (0)** di seluruh posisinya, kecuali satu posisi **tunggal (hot)** bernilai **satu (1)** yang mewakili kata tersebut.

Setiap indeks posisi mewakili satu kata unik yang telah dipetakan dalam kamus data.

Contoh 1: Korpus 6 Kata Unik



Dokumen Input Korpus:

Doc 1: *"I love playing football."* | Doc 2: *"Indians love playing Cricket."*



Ukuran Kosakata (N = 6):

```
{'i': 0, 'love': 1, 'playing': 2, 'football': 3, 'indians': 4, 'cricket': 5}
```



Representasi Word Index:

```
Doc 1: [0, 1, 2, 3] | Doc 2: [4, 1, 2, 5]
```

Contoh 1: Vektor One-Hot (Dimensi 6)

Kata Unik	Indeks	Vektor One-Hot (Dimensi N = 6)	Muncul di
i	0	[1, 0, 0, 0, 0, 0]	Doc 1
love	1	[0, 1, 0, 0, 0, 0]	Doc 1 & Doc 2
playing	2	[0, 0, 1, 0, 0, 0]	Doc 1 & Doc 2
football	3	[0, 0, 0, 1, 0, 0]	Doc 1
indians	4	[0, 0, 0, 0, 1, 0]	Doc 2
cricket	5	[0, 0, 0, 0, 0, 1]	Doc 2

Contoh 2: Korpus 21 Kata Unik (3 Dokumen)

Dokumen Korpus Input

Doc 1: *"My name is John, What is your name?"*

Doc 2: *"Bill is a very good person. He likes playing soccer."*

Doc 3: *"What is your favorite game? I love Football. Football is a great game."*

Pemetaan Kosakata (N = 21)

Korpus ini mengekstrak total **21 kata unik** (indeks 0 hingga 20):

```
{'my': 0, 'name': 1, 'is': 2, 'john': 3, 'what': 4, 'your': 5, 'bill': 6, 'a': 7, 'very': 8, 'good': 9, 'person': 10, 'he': 11, 'likes': 12, 'playing': 13, 'soccer': 14, 'favorite': 15, 'game': 16, 'i': 17, 'love': 18, 'football': 19, 'great': 20}
```

Contoh 2: Representasi Word Index & Vektor



Representasi Urutan Indeks Kata (Word Index):

```
Doc 1: [0, 1, 2, 3, 4, 2, 5, 1]  
Doc 2: [6, 2, 7, 8, 9, 10, 11, 12, 13, 14]  
Doc 3: [4, 2, 5, 15, 16, 17, 18, 19, 19, 2, 7, 20, 16]
```



Struktur Vektor One-Hot (Dimensi N = 21):

```
'my'      = [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]  
'name'    = [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]  
'great'   = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
```

Keterbatasan One-Hot Encoding



1. Konsumsi Memori Tinggi

Seiring membesarnya ukuran kosakata (ribuan/jutaan kata), kebutuhan memori melonjak drastis karena sebagian besar nilai bernilai nol (**sparse matrix**).



2. Curse of Dimensionality

Ruang fitur bertambah eksplosif seiring penambahan kata baru, membebankan komputasi algoritma pembelajaran mesin (ML).



3. Tanpa Informasi Semantik

Setiap kata dianggap ortogonal (berjarak sama). Kata berdekatan seperti "**football**" dan "**soccer**" tidak memiliki indikator kemiripan makna.



Kegunaan Utama

One-Hot Encoding tetap efektif untuk dataset kecil, variabel kategorikal, dan model baseline NLP yang sederhana.



Word Embeddings

Teknik modern seperti **Word2Vec** & **GloVe** memetakan kata ke dimensi padat (dense) untuk menangkap konteks dan kemiripan semantik.



Model Transformer

Arsitektur canggih (BERT/GPT) menggunakan representasi kontekstual dinamis untuk mengatasi kelemahan encoding tradisional.

Model Bag-of-Words (BOW)

Apa itu Bag-of-Words (BOW)?

Metode representasi teks paling sederhana dalam NLP yang menghitung frekuensi kemunculan setiap kata di dalam dokumen.

Prinsip Kerja Utamanya

- Menerapkan Label / Integer Word Encoding.
- Mengabaikan tata bahasa (grammar) dan urutan kata.
- Jumlah frekuensi kata dianggap sebagai bobot ('weightage') kata tersebut.

Penggunaan Utama di Industri

- **Mesin Pencari (Search):**

Mencocokkan kata kunci pencarian dengan dokumen berdasarkan frekuensi kata.

- **Sistem Rekomendasi:**

Merekomendasikan artikel atau produk yang mirip berdasarkan kesamaan konten teks.

- **Klasifikasi Teks:**

Sebagai input ekstraksi fitur awal untuk model Machine Learning (Spam detection, Analisis Sentimen).

Contoh 1: Vektor BOW Sederhana (2 Dokumen)

Korpus Dokumen (N = 2):

- Doc 1: "I love playing football."
- Doc 2: "Indians love playing Cricket."

Kamus Kosakata (6 Kata Unik):

Indeks 0 : 'i'
Indeks 1 : 'love'
Indeks 2 : 'playing'
Indeks 3 : 'football'
Indeks 4 : 'indians'
Indeks 5 : 'cricket'

Hasil Vektor BOW (Panjang Vektor = 6):

- BOW Doc 1:
[1, 1, 1, 1, 0, 0]
(Ada 1 kata 'i', 'love', 'playing', 'football')
- BOW Doc 2:
[0, 1, 1, 0, 1, 1]
(Ada 1 kata 'love', 'playing', 'indians', 'cricket')

Contoh 2: Vektor BOW pada Korpus Lebih Besar

Korpus Dokumen (N = 3):

- Doc 1: "My name is John, What is your name?"
- Doc 2: "Bill is a very good person. He likes playing soccer."
- Doc 3: "What is your favorite game? I love Football. Football is a great game."

Ukuran Kamus Kosakata (21 Kata Unik / Indeks 0 s.d. 20):

{0: 'my', 1: 'name', 2: 'is', 3: 'john', 4: 'what', 5: 'your', 6: 'bill', 7: 'a', 8: 'very', 9: 'good', 10: 'person', 11: 'he', 12: 'likes', 13: 'playing', 14: 'soccer', 15: 'favorite', 16: 'game', 17: 'i', 18: 'love', 19: 'football', 20: 'great'}

Vektor Frekuensi BOW (Ukuran Vektor = 21):

- BOW Doc 1: [1, 2, 2, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] --> ('name':2, 'is':2, dst.)
- BOW Doc 2: [0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0] --> ('is':1, 'bill':1, dst.)
- BOW Doc 3: [0, 0, 2, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 2, 1, 1, 2, 1] --> ('is':2, 'game':2, 'football':2, dst.)

Keterbatasan Utama Bag-of-Words

- **Kehilangan Urutan Kata:**

Urutan kata dan konteks kalimat diabaikan sepenuhnya ('not good' dianggap sama dengan 'good').

- **Dominasi Kata Umum:**

Kata-kata umum yang muncul sangat sering (seperti 'is', 'a', 'the') mendapat bobot paling tinggi padahal kurang informatif.

- **Sparsity (Matriks Jarang):**

Jika jumlah dokumen & kosakata besar, mayoritas elemen matriks bernilai 0 sehingga boros memori.

- **Masalah OOV (Out of Vocabulary):**

Tidak dapat mengenali kata baru yang tidak ada pada saat pembuatan kamus kosakata.

Solusi & Evolusi Model NLP

- **TF-IDF (Term Frequency - Inverse Document Frequency):**

Memperbaiki BOW dengan memberikan penalti/diskon pada kata-kata umum yang sering muncul di banyak dokumen.

- **N-Gram (Bi-gram, Tri-gram):**

Menggabungkan N kata berurutan untuk mempertahankan sebagian konteks lokal.

- **Word Embeddings (Word2Vec, GloVe):**

Representasi vektor padat (dense) yang mampu menangkap kemiripan makna/semantik antar kata.

- **Transformers (BERT, GPT):**

Model modern berbasis perhatian (Attention) yang memahami konteks secara penuh dalam kalimat.

TF-IDF

Apa itu TF-IDF?

TF-IDF (Term Frequency - Inverse Document Frequency) adalah metode pembobotan teks populer dalam NLP untuk Mesin Pencari (Search Engine), Sistem Rekomendasi, dan Klasifikasi Teks.

Prinsip Kerja Utama

- **Pentingnya Kata:** Memberikan bobot tinggi jika kata muncul berkali-kali dalam dokumen tertentu.
- **Penalti Kata Umum:** Mengurangi bobot secara drastis jika kata muncul terlalu sering di banyak dokumen.

Mengapa Dibuat (Mengatasi Kelemahan BOW)?

Metode Bag of Words (BOW) memiliki kelemahan: pembobotan didominasi oleh kata-kata umum (seperti 'is', 'the', 'a', 'dan') yang sering muncul tetapi tidak membawa informasi spesifik.

Penggunaan Utama di Industri

1. **Mesin Pencari:** Pemeringkatan dokumen berdasarkan relevansi kata kunci.
2. **Sistem Rekomendasi:** Pencocokan artikel berdasarkan kemiripan isi.
3. **Klasifikasi Teks:** Ekstraksi fitur untuk model Machine Learning.

Formulasi Matematika TF-IDF

$$\text{tf_idf}(t, d, D) = \text{tf}(t, d) \times \text{idf}(t, D)$$

1. Term Frequency (tf)

$$\text{tf}(t, d) = \text{jumlah } t \text{ dalam } d$$

Meningkat secara proporsional seiring makin seringnya kata muncul di dalam dokumen tersebut.

2. Inverse Document Frequency (idf)

$$\text{idf}(t) = \log(N / \text{df}(t))$$

Nilainya menurun seiring meningkatnya jumlah dokumen yang memuat kata tersebut di seluruh korpus.

Glosarium Variabel

- tf : Frekuensi kata/istilah dalam dokumen spesifik
- idf : Kebalikan frekuensi dokumen dalam korpus
- t : Istilah / Kata (Term)
- d : Dokumen spesifik
- D : Seluruh koleksi dokumen (Korpus)
- N : Jumlah total dokumen di dalam korpus
- df(t) : Jumlah dokumen yang mengandung kata t

Contoh 1: Persiapan Korpus & Vektor TF

Data Dokumen Input (Korpus, N = 2):

- Doc 1: "I love playing football."
- Doc 2: "Indians love playing Cricket."

Kamus / Vocab (6 Kata Unik):

Indeks 0 : 'i'
Indeks 1 : 'love'
Indeks 2 : 'playing'
Indeks 3 : 'football'
Indeks 4 : 'indians'
Indeks 5 : 'cricket'

Matriks Term Frequency (tf):

Vektor Frekuensi Kata (seperti Bag of Words):

- Doc 1: [1, 1, 1, 1, 0, 0]
('i':1, 'love':1, 'playing':1, 'football':1)
- Doc 2: [0, 1, 1, 0, 1, 1]
('love':1, 'playing':1, 'indians':1, 'cricket':1)

Contoh 1: Perhitungan IDF & Bobot TF-IDF

Perhitungan IDF: $\log(N / df)$

- $\text{idf}('i') [df=1] = \log(2/1) = 0.6931$
- $\text{idf}('love') [df=2] = \log(2/2) = 0.0000$ (Penalti!)
- $\text{idf}('playing') [df=2] = \log(2/2) = 0.0000$ (Penalti!)
- $\text{idf}('football') [df=1] = \log(2/1) = 0.6931$
- $\text{idf}('indians') [df=1] = \log(2/1) = 0.6931$
- $\text{idf}('cricket') [df=1] = \log(2/1) = 0.6931$

Matriks Bobot Akhir TF-IDF ($tf \times idf$)

Doc 1:

[0.693, 0.000, 0.000, 0.693, 0.000, 0.000]

Doc 2:

[0.000, 0.000, 0.000, 0.000, 0.693, 0.693]

Insight Kunci:

Kata 'love' dan 'playing' diberi bobot 0.000 karena muncul di semua dokumen. TF-IDF berhasil meredam kata umum dan menyoroti kata kunci spesifik seperti 'football' dan 'cricket'.

Keunggulan Utama

- Eliminasi Kata Umum Otomatis: Meredam stop words tanpa pembersihan manual.
- Menyoroti Kata Kunci: Skor tinggi untuk kata yang unik dan mencirikan dokumen.
- Efisien & Sederhana: Cepat dihitung secara komputasi.
- Bagus sebagai Baseline: Sangat cocok untuk model awal klasifikasi teks.

Keterbatasan & Solusi Modern

- Agnostik Konteks: Tidak memperhitungkan urutan kata atau makna semantik.
- Sparse Matrix: Didominasi nilai nol jika kosakata sangat besar.
- Masalah OOV: Tidak bisa menangani kata baru di luar data latih.

Solusi Modern NLP:

Evolusi berlanjut ke Word Embeddings (Word2Vec, GloVe) & Transformer (BERT, GPT) yang mampu memahami konteks dan makna kata.

SVD Based Word Embedding

Mengapa Membutuhkan Word Embedding?

Keterbatasan Encoding Tradisional

- Tanpa Hubungan Semantik: Integer/One-Hot Encoding tidak dapat mendeteksi bahwa kata 'Soccer' dan 'Football' memiliki makna serupa.
- Curse of Dimensionality: Ukuran matriks One-Hot membengkak seiring bertambahnya kosakata.
- Bias Tak Diinginkan: Integer Label Encoding memberikan bobot urutan numerik acak yang salah diartikan oleh model.

Solusi: Representasi Vector Embedding

- **Definisi Embedding:**

Representasi kata dalam ruang vektor berdimensi tetap (misal: $N = 100$ hingga 300) yang independen dari ukuran total kosakata.

Document-Term Matrix & Reduksi Dimensi

2 Metode Utama Generasi Embedding

1. Metodologi Berbasis SVD (Matrix Factorization)

Membuat matriks ko-okurensi (Co-occurrence Matrix) terlebih dahulu, lalu mereduksi dimensinya menggunakan Singular Value Decomposition (SVD).

2. Metodologi Berbasis Neural Network / Iterasi

Mempelajari vektor kata secara langsung dengan memprediksi kata sekitar melalui model iteratif (misal: Word2Vec CBOW & Skip-Gram).

Dua Cara Membuat Matriks Ko-Okurensi SVD:

- Document-Term Matrix (DTM)
- Window-based Co-Occurrence Matrix

Konsep Document-Term Matrix (DTM)

• Pembentukan Matriks Awal:

Menghitung frekuensi kemunculan setiap kata di setiap dokumen. Menghasilkan matriks berukuran $(N \times D)$ di mana N = Jumlah Kosakata, D = Jumlah Dokumen.

• Menerapkan SVD (Singular Value Decomposition):

Matriks $(N \times D)$ ditransformasikan & direduksi menjadi matriks $(N \times d)$, di mana d adalah dimensi embedding yang diinginkan (misal $d = 2$ atau 100 s.d. 500).

• Hasil Vektor Kata:

Setiap baris pada matriks hasil SVD menjadi embedding berdimensi- d untuk kata bersangkutan.

Contoh Transformasi DTM ke Embedding 2D via SVD

Korpus Dokumen Input (N = 15 kata unik, D = 3 dokumen):

- Doc 1: "I love playing both sports, Cricket and Football"
- Doc 2: "Indians play both sports, Cricket and Football"
- Doc 3: "Football is more popular sport than Cricket"

Word Index	Word	Doc1 Occurrence	Doc2 Occurrence	Doc3 Occurrence
0	i	1	0	0
1	love	1	0	0
2	playing	1	0	0
3	both	1	1	0
4	sports	1	1	0
5	cricket	1	1	1
6	and	1	1	0
7	football	1	1	1
8	indians	0	1	0
9	play	0	1	0
10	is	0	0	1
11	more	0	0	1
12	popular	0	0	1
13	sport	0	0	1
14	than	0	0	1

Idx	Word	Dim 1	Dim 2
0	i	0.6801	-0.3136
1	love	0.6801	-0.3136
2	playing	0.6801	-0.3136
3	both	1.3022	-0.5450
4	sports	1.3022	-0.5450
5	cricket	1.6901	0.3759
6	and	1.3022	-0.5450
7	football	1.6901	0.3759
8	indians	0.6221	-0.2314
9	play	0.6221	-0.2314
10	is	0.3879	0.9209
11	more	0.3879	0.9209
12	popular	0.3879	0.9209
13	sport	0.3879	0.9209
14	than	0.3879	0.9209

Masalah one hot encoding

1. Tidak Ada Hubungan Semantik

Metode tradisional tidak dapat menangkap kemiripan makna antar kata. Kata seperti "**Sepak Bola**" dan "**Futsal**" diperlakukan sebagai entitas terpisah yang sepenuhnya independen (ortogonal).

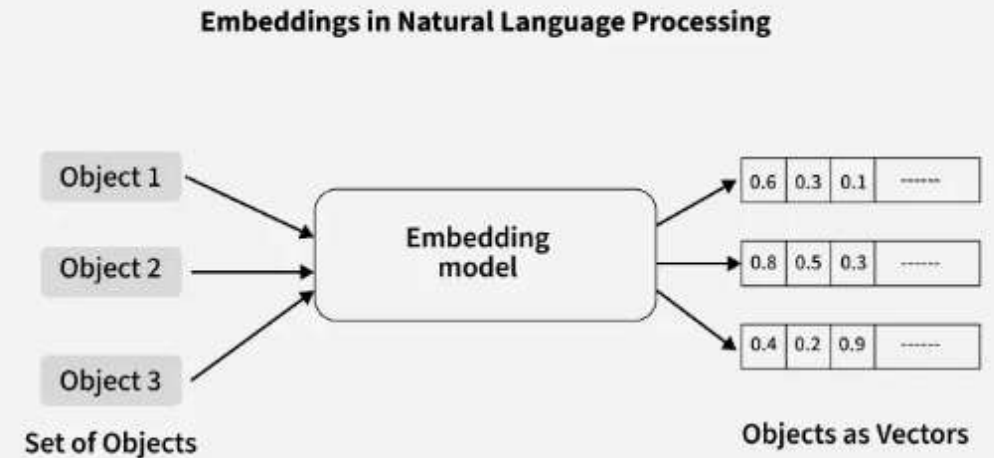
2. Ledakan Dimensi (Curse of Dimensionality)

Pada One-Hot Encoding, ukuran vektor bertambah seiring membesarnya kosakata. Untuk 1 juta kata, setiap kata membutuhkan vektor berukuran 1 juta elemen, sangat tidak efisien dalam memori.

Apa Itu Word Embeddings?

Word Embeddings adalah teknik pemetaan kata tunggal ke dalam vektor berdimensi rendah bernilai riil (biasanya 50 hingga 300 dimensi).

- ✓ **Dimensi Tetap:** Ukuran vektor independen dari total jumlah kosakata dalam korpus.
- ✓ **Jarak Vektor Semantik:** Kemiripan kata diukur langsung melalui jarak spasial (misal: Cosine Similarity).
- ✓ **Mengatasi High-Dimensionality:** Memangkas matriks jarang (sparse) menjadi matriks padat (dense).



| Intuisi Utama Word Embeddings

“

"You shall know a word by the company it keeps."

”

— J. R. Firth (1957)

Makna sebuah kata ditentukan oleh kata-kata tetangga yang sering muncul di sekitarnya dalam korpus teks.

Metodologi Generasi Embedding

1. Metode Berbasis SVD

Menggunakan faktorisasi matriks **Singular Value Decomposition (SVD)** pada matriks kemunculan bersama (Co-occurrence Matrix) untuk reduksi dimensi.

2. Metode Berbasis Neural Network

Menggunakan model iteratif seperti **Word2Vec (CBOW & Skip-Gram)** untuk memprediksi kata tetangga secara langsung dari korpus besar.

Keterbatasan Metode Berbasis SVD



1. Ketidakseimbangan Matriks

Kata-kata berfrekuensi tinggi (seperti "dan", "yang") mendominasi matriks, menyebabkan skewness data yang parah.



2. Biaya Komputasi Tinggi

Kompleksitas perhitungan SVD sangat mahal, yaitu mencapai:

$$O(m \cdot n^2)$$

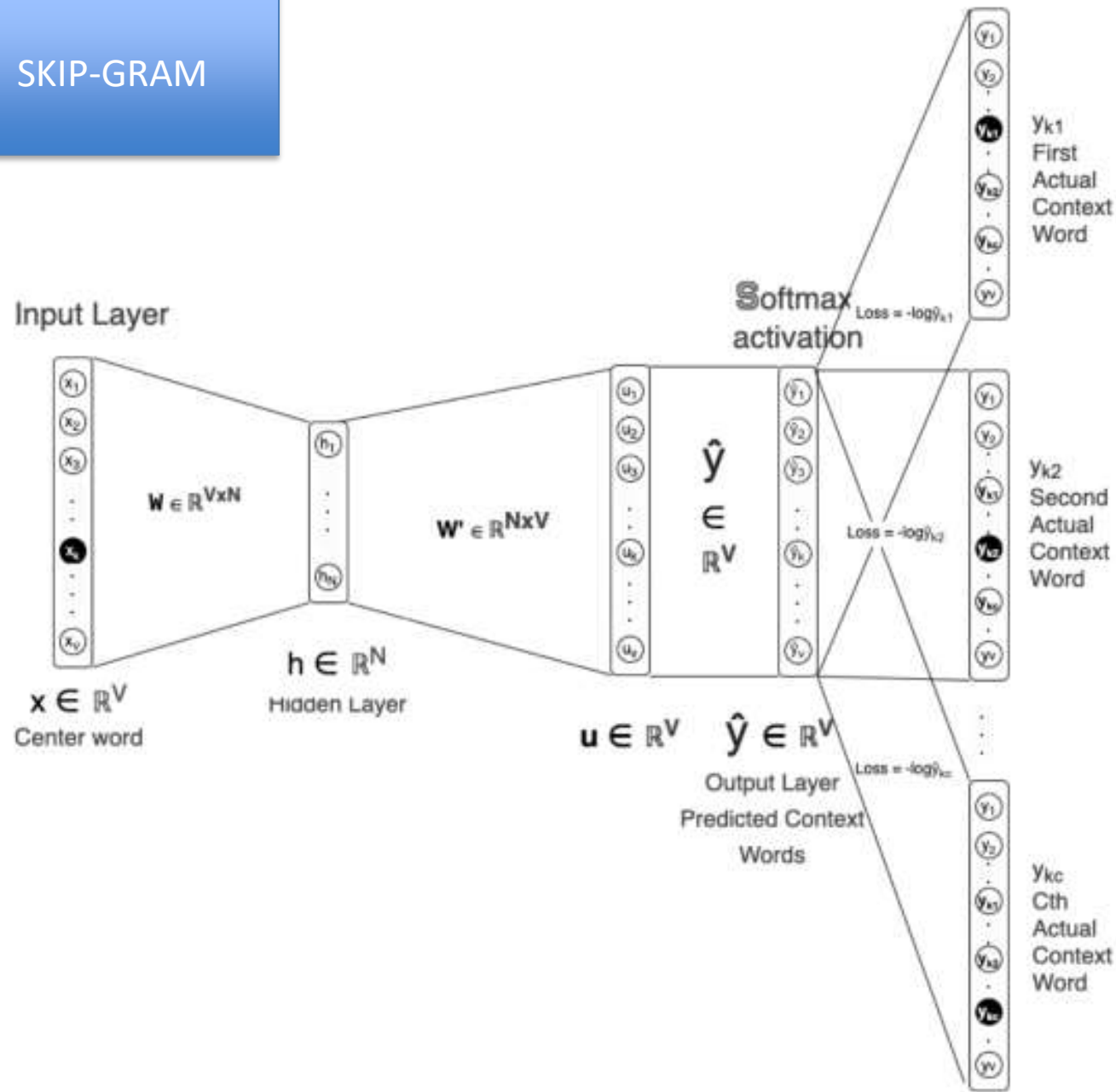
di mana m adalah ukuran kosakata dan n adalah dimensi embedding.



3. Sulit Diperbarui

Ketika ada kata atau dokumen baru ditambahkan, seluruh matriks harus dihitung ulang dari awal.

SKIP-GRAM



SYMBOLS MEANING:

x : input layer, which is one-hot encoding of center word
 W : weight between the input layer and hidden layer
 W' : weight between the hidden layer and output layer
 V : Vocabulary Size
 N : Embedding Size/ Size of hidden layer
 h : the hidden layer
 u : Output layer before activation
 $\hat{y}$: Output layer after activation, predicted context words
 y : Actual context words for center word x
 α : Learning rate
 C : Number of context words for a given center word
 T : Number of training examples

FORWARD PROPAGATION:

$$h = W^T \cdot x$$

$$u = W'^T \cdot h$$

$$\hat{y} = \text{softmax}(u)$$

Loss for every context word c for a given center word

$$J = - \sum_{w \in V} y_w \cdot \log(\hat{y}_w) = -\log(\hat{y}_c)$$

Loss for all context words C for a given center word is as following

$$J = - \sum_{c=1}^C \log(\hat{y}_c)$$

We have T training examples and C context words, the objective is to minimize following loss function:

$$J = -\frac{1}{T} \sum_{t=1}^T \sum_{c=1}^C \log(\hat{y}_{t,c})$$

BACKWARD PROPAGATION:

$$\partial J / \partial w'_{i,j} = \sum_{c=1}^C (y_{c,j} - t_{c,j}) \cdot h_i$$

$$\partial J / \partial w_{k,i} = \sum_{j=1}^V \sum_{c=1}^C (y_{c,j} - t_{c,j}) \cdot w'_{i,j} \cdot x_k$$

where $t_{c,j} = 1$ if the j^{th} node of the c^{th} true output is 1 else $t_{c,j} = 0$

$$w'_{i,j}^{(new)} = w'_{i,j}^{(old)} - \alpha \cdot \sum_{c=1}^C (y_{c,j} - t_{c,j}) \cdot h_i$$

$$w_{k,i}^{(new)} = w_{k,i}^{(old)} - \alpha \cdot \sum_{j=1}^V \sum_{c=1}^C (y_{c,j} - t_{c,j}) \cdot w'_{i,j} \cdot x_k$$

Pelatihan Model Word Embedding SKIP-GRAM

Parameter & Data Pelatihan

Parameter Utama

- ✓ Ukuran embedding $N = 2$
- ✓ Ukuran kosakata $V = 6$
- ✓ Ukuran jendela (*window size*) = 1

Kalimat Sampel

"I love playing Football"
"I love playing Cricket"
"I love playing sports"

Pembentukan Data

Memetakan setiap **kata pusat** (*center word*) dengan **kata konteks** (*context word*) di sekitarnya berdasarkan ukuran jendela 1.

Arsitektur Jaringan Saraf

🎯 Tujuan Model

Memprediksi kata konteks berdasarkan kata pusat yang diberikan.

🏗️ Dimensi Lapisan Bobot

Lapisan pertama: $\text{Ukuran Kosakata} \times \text{Ukuran Embedding}$

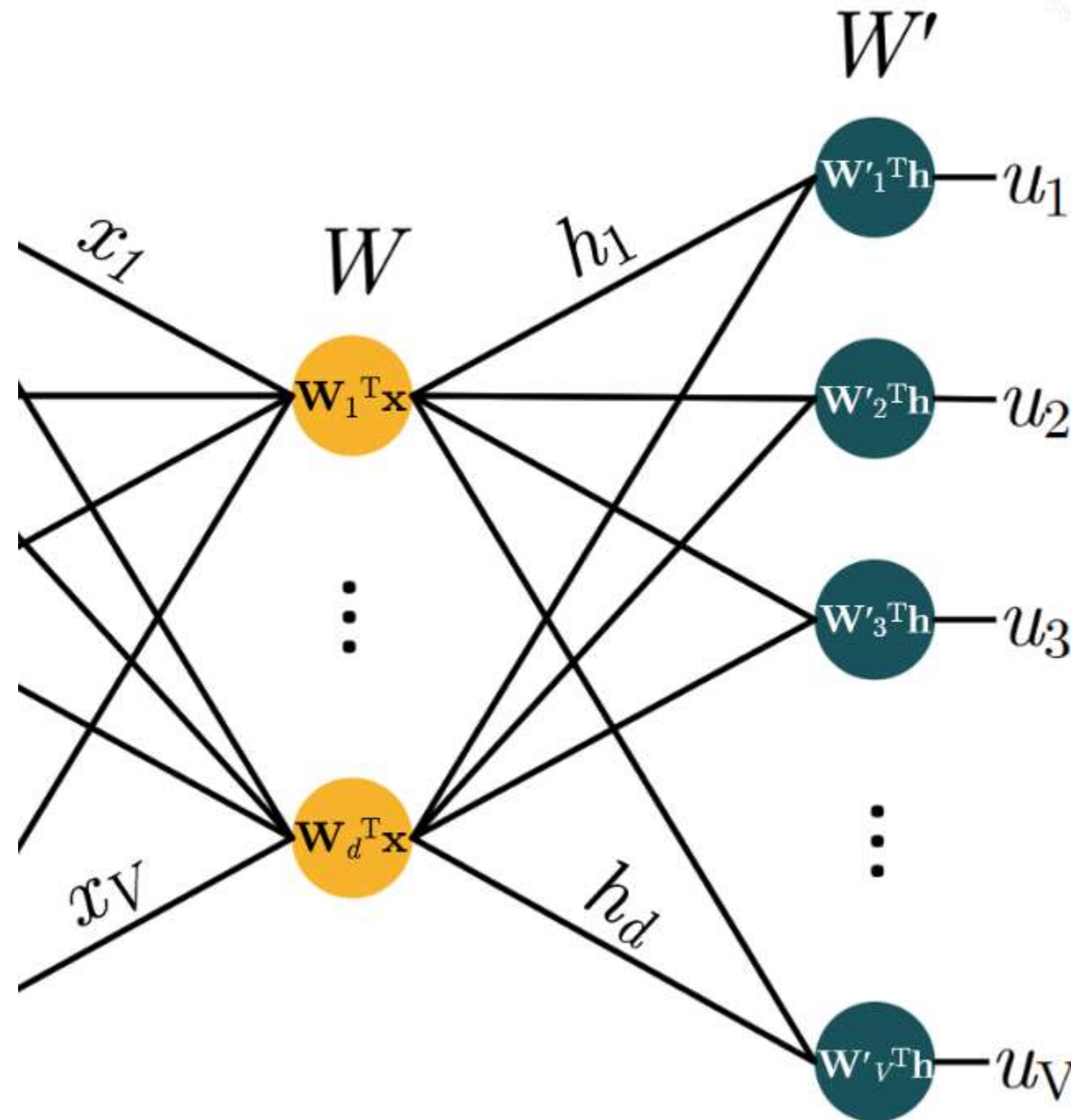
Lapisan kedua: $\text{Ukuran Embedding} \times \text{Ukuran Kosakata}$

📈 Prediksi & Output

Lapisan akhir menggunakan fungsi aktivasi **softmax** untuk menghasilkan prediksi kata konteks.

🚩 Hasil Akhir

Setiap baris bobot pada lapisan pertama menjadi *embedding* bagi kata yang sesuai setelah pelatihan selesai.



Pengodean Kosakata & Vektor Pelatihan

Indeks Kosakata

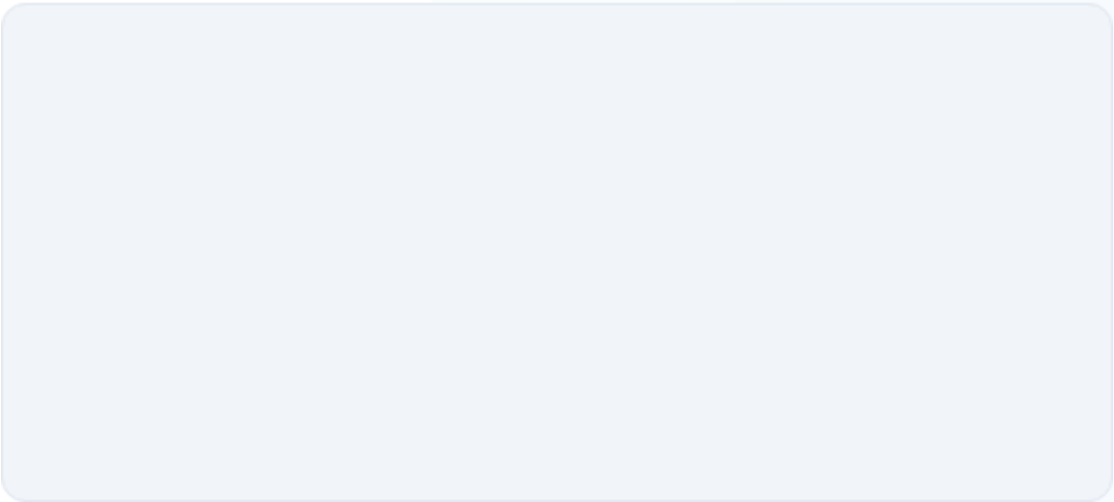
Daftar unik token beserta nilai indeksnya (Ukuran Kosakata $V = 6$).

Indeks	Kata (Token)
0	'I'
1	'love'
2	'playing'
3	'football'
4	'cricket'
5	'sports'

Pengodean Data Pelatihan

Transformasi teks menjadi format numerik untuk proses komputasi *machine learning*:

- ✓ X_{train} : Representasi vektor *one-hot* untuk **kata pusat**.
- ✓ y_{train} : Representasi vektor target (*one-hot*) untuk **kata konteks**.



We are considering **Embedding_size N** as **2** in our example.

Vocabulary size V in the above example sentences is **6**.

In our example sentence, our training data will look like the following with the window size 1:

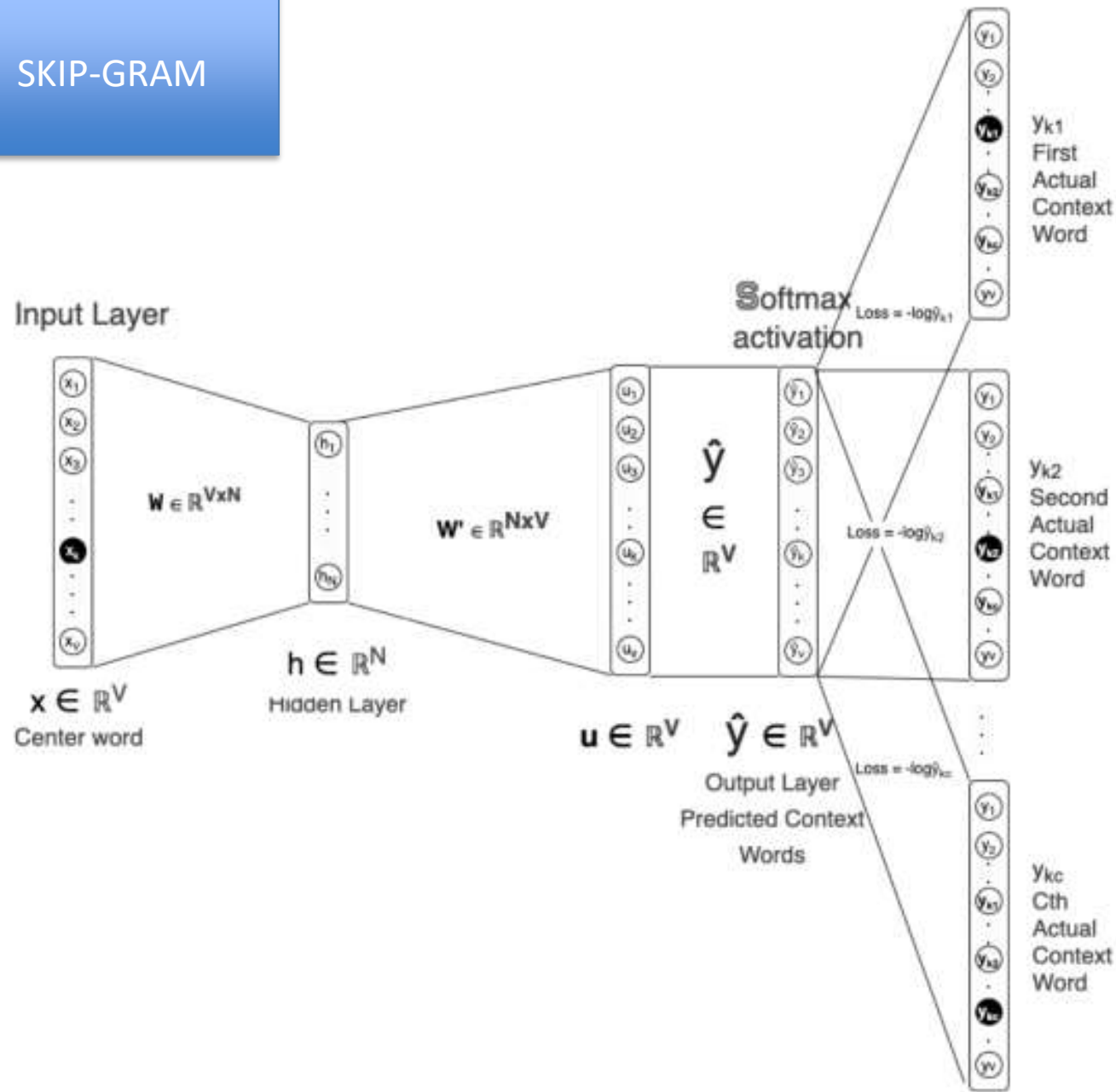
<u>Sentence</u>	<u>Center Word</u> (<u>X_center_word_train</u>)	<u>Context</u> <u>Words (y_context_words_train)</u>
Sentence1: I love playing Football	I	love
Sentence1: I love playing Football	love	I, Playing
Sentence1: I love playing Football	playing	love, Football
Sentence1: I love playing Football	Football	playing
Sentence2: I love playing Cricket	I	love
Sentence2: I love playing Cricket	love	I, playing
Sentence2: I love playing Cricket	playing	love, Cricket
Sentence2: I love playing Cricket	Cricket	playing
Sentence3: I love playing sports	I	love
Sentence3: I love playing sports	love	I, playing
Sentence3: I love playing sports	playing	love, sports
Sentence3: I love playing sports	sports	playing

<u>Sentence</u>	<u>Center Word (X_train)</u>	<u>Context Words (y_train)</u>
Sentence1: I love playing Football	I [1, 0, 0, 0, 0, 0]	love [0, 1, 0, 0, 0, 0]
Sentence1: I love playing Football	love [0, 1, 0, 0, 0, 0]	I, Playing [1, 0, 1, 0, 0, 0]
Sentence1: I love playing Football	playing [0, 0, 1, 0, 0, 0]	love, Football [0, 1, 0, 1, 0, 0]
Sentence1: I love playing Football	Football [0, 0, 0, 1, 0, 0]	playing [0, 0, 1, 0, 0, 0]
Sentence2: I love playing Cricket	I [1, 0, 0, 0, 0, 0]	love [0, 1, 0, 0, 0, 0]
Sentence2: I love playing Cricket	love [0, 1, 0, 0, 0, 0]	I, playing [1, 0, 1, 0, 0, 0]
Sentence2: I love playing Cricket	playing [0, 0, 1, 0, 0, 0]	love, Cricket [0, 1, 0, 0, 1, 0]
Sentence2: I love playing Cricket	Cricket [0, 0, 0, 0, 1, 0]	playing [0, 0, 1, 0, 0, 0]
Sentence3: I love playing sports	I [1, 0, 0, 0, 0, 0]	love [0, 1, 0, 0, 0, 0]
Sentence3: I love playing sports	love [0, 1, 0, 0, 0, 0]	I, playing [1, 0, 1, 0, 0, 0]
Sentence3: I love playing sports	playing [0, 0, 1, 0, 0, 0]	love, sports [0, 1, 0, 0, 0, 1]
Sentence3: I love playing sports	sports [0, 0, 0, 0, 0, 1]	playing [0, 0, 1, 0, 0, 0]

X_train: [[1, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], [0, 0, 0, 1, 0, 0], [1, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0], [1, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 1]]

y_train: [[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 0, 0], [0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 0, 1, 0], [0, 0, 1, 0, 0, 0], [0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 0, 0, 1], [0, 0, 1, 0, 0, 0]]

SKIP-GRAM



SYMBOLS MEANING:

x : input layer, which is one-hot encoding of center word
 W : weight between the input layer and hidden layer
 W' : weight between the hidden layer and output layer
 V : Vocabulary Size
 N : Embedding Size/ Size of hidden layer
 h : the hidden layer
 u : Output layer before activation
 $\hat{y}$: Output layer after activation, predicted context words
 y : Actual context words for center word x
 α : Learning rate
 C : Number of context words for a given center word
 T : Number of training examples

FORWARD PROPAGATION:

$$h = W^T \cdot x$$

$$u = W'^T \cdot h$$

$$\hat{y} = \text{softmax}(u)$$

Loss for every context word c for a given center word

$$J = - \sum_{w \in V} y_w \cdot \log(\hat{y}_w) = -\log(\hat{y}_c)$$

Loss for all context words C for a given center word is as following

$$J = - \sum_{c=1}^C \log(\hat{y}_c)$$

We have T training examples and C context words, the objective is to minimize following loss function:

$$J = -\frac{1}{T} \sum_{t=1}^T \sum_{c=1}^C \log(\hat{y}_{t,c})$$

BACKWARD PROPAGATION:

$$\partial J / \partial w'_{i,j} = \sum_{c=1}^C (y_{c,j} - t_{c,j}) \cdot h_i$$

$$\partial J / \partial w_{k,i} = \sum_{j=1}^V \sum_{c=1}^C (y_{c,j} - t_{c,j}) \cdot w'_{i,j} \cdot x_k$$

where $t_{c,j} = 1$ if the j^{th} node of the c^{th} true output is 1 else $t_{c,j} = 0$

$$w'_{i,j}^{(new)} = w'_{i,j}^{(old)} - \alpha \cdot \sum_{c=1}^C (y_{c,j} - t_{c,j}) \cdot h_i$$

$$w_{k,i}^{(new)} = w_{k,i}^{(old)} - \alpha \cdot \sum_{j=1}^V \sum_{c=1}^C (y_{c,j} - t_{c,j}) \cdot w'_{i,j} \cdot x_k$$

<u>Word Index</u>	<u>Word</u>	<u>Embedding-></u>	
0	i	1.149123999	-0.8699841668
1	love	-0.04047703988	1.483339811
2	playing	0.5546634793	-0.8665633952
3	football	0.02787686912	0.2541694707
4	cricket	-0.06727488884	0.2959596391
5	sports	-0.03852687391	0.3858703899